



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$10 ủy thác - Delegate, Sự kiện - event





Nội dung

- ủy thác
- Sự kiện





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

1. Ủy thác - Delegate



Vấn đề/bài toán

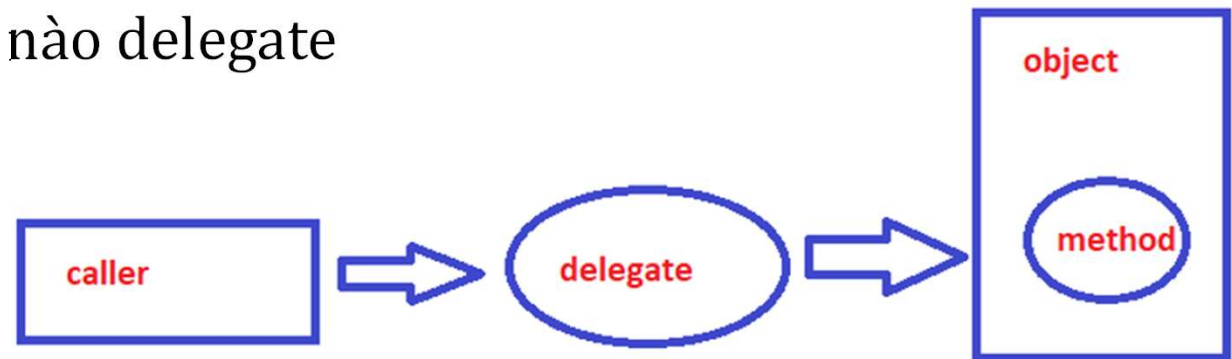
- Callback:
 - Khi nào học xong => chơi game
 - Tải xong => thông báo
 - Đoạn code B thực thi, đến lúc nào đó cần gọi tác vụ A

ủy thác - delegate

- Đóng gói phương thức (và cả đối tượng) vào đối tượng delegate
- Là lớp đối tượng (class)
 - Có thể hiện instance
 - Có thể chứa tham chiếu tới 1 hay nhiều method

Delegate trong C# tương tự con trỏ hàm trong C++

Java: cũng có hỗ trợ phần nào delegate



ủy thác

- Delegate là một kiểu đối tượng biểu diễn tham chiếu tới Phương thức với danh sách tham số và kiểu trả về
- Mỗi delegate định nghĩa một chữ ký Phương thức
 - danh sách tham số
 - kiểu trả về
- Mọi method cùng chữ ký => đều có thể thêm vào (add) thể hiện của delegate
- Mỗi đối tượng delegate có danh sách các tham chiếu method
 - thêm - add (+)
 - loại bỏ - remove (-)

Thực thi delegate

- Định nghĩa delegate

- Từ khóa delegate

- `delegate` <return> <delegate_name> (<list_param>)

```
// Define a delegate
```

```
public delegate int Operation(int x, int y);
```

- VD trên: khai báo delegate cho các method trả về int, nhận 2 đối số int

```
int anyMethod(int x, int y);
```

Thực thi delegate

- Lớp đối tượng có method khớp với delegate

```
// Class with methods that match the delegate signature
public class Calculator
{
    public int Add(int x, int y){
        return x + y;
    }

    public int Subtract(int x, int y) {
        return x - y;
    }
}
```

Thực thi delegate

■ Thực thi

- Khởi tạo delegate, gán method cho delegate
- Gọi delegate

```
public static void Main()
{
    Calculator calculator = new Calculator();
    // Khởi tạo và gán method cho delegate
    Operation operation = new Operation(calculator.Add);
    // thực thi
    Console.WriteLine($"Addition: {operation(5, 3)}"); //console output: 8
    operation = new Operation(calculator.Subtract);
    Console.WriteLine($"Subtraction: {operation(5, 3)}"); // console output: 2
}
```

Combine delegate - Multicast

- Cho phép nhiều đối tượng (method) gán cho cùng một đối tượng delegate => một lần gọi delegate sẽ thực thi tất cả các method
- Thiết lập bao đóng các method
 - Thêm (+) method vào delegate: `someDelegate += method1`
 - Loại bỏ (-) method khỏi delegate: `someDelegate -= method1`

Combine delegate - Multicast

- VD: gửi thông báo qua nhiều kênh

```
// Define a delegate
public delegate void NotifyDelegate();

public class Notifier
{
    public void SendEmail() {
        Console.WriteLine("Email sent.");
    }
    public void SendSMS() {
        Console.WriteLine("SMS sent.");
    }
}
```

Combine delegate - Multicast

- VD: gửi thông báo qua nhiều kênh

```
public static void Main()
{
    Notifier notifier = new Notifier();
    // Create a multicast delegate
    NotifyDelegate notify = new NotifyDelegate();
    notify += notifier.SendEmail;
    notify += notifier.SendSMS;
    // Invoke the multicast delegate
    notify();
}

// Console output:
// Email Sent
// SMS Sent
```

VD Bài toán sắp xếp

- Xây dựng ứng dụng sắp xếp array với phần tử là đối tượng bất kỳ
- Vấn đề: cần thủ tục so sánh 2 phần tử
 - Nếu phần tử là số $\Rightarrow$ phép toán $+/-$
 - Nếu phần tử loại đối tượng khác ?

VD Bài toán sắp xếp

■ Giải pháp

- Đối tượng tự cài đặt thủ tục so sánh
- Sắp xếp: sử dụng delegate để gọi tới thủ tục so sánh phù hợp

```
void Sort(object[] list, CompareObject cmp)
```

Delegate này tham chiếu tới hàm Compare của lớp MyClass
(Chính MyClass sẽ tự xác định thứ tự các đối tượng)

VD Bài toán sắp xếp

- Delegate

```
public delegate bool CompareObject(object obj1, object obj2)
```

Trả về true: nếu obj1 “trước” obj2
false: ngược lại

- Class: tự cài đặt Phương thức so sánh

```
class Person {  
    private string name;  
    private DateTime yearOfBirth;  
    public static bool CompareByName(object p1, object p2) {  
        if (string.Compare(((Person)p1).name, ((Person)p2).name) < 0)  
            return true;  
        return false;  
    }  
}
```

Ủy thác để làm gì ?

- Gọi hàm một cách linh hoạt
- Tạo cơ chế callback (kết thúc => gọi hàm trả về) - event driven

Delegate

Action, Predicate, Fun

- Action: `Action<T in1, T in2, ...>`.
 - Action tương đương 1 delegate với kiểu trả về là void, với in1, in2, ... là các params nhận vào.
- Predicate: `Predicate<T in>`
 - Predicate tương đương 1 delegate với kiểu trả về là bool, với in là các param nhận vào. Predicate chỉ có thể nhận vào 1 param duy nhất.
- Func: `Func<T in1, T in2, ... , T result>`
 - Function tương đương 1 delegate với kiểu trả về (result); in1, in2, ... là các params nhận vào
 - Func bắt buộc phải trả ra giá trị, không thể là void

Delegate

Action, Predicate, Fun

■ VD

```
public static void ChoiGame(string game) {  
    Console.WriteLine(game);  
}
```

```
void Main(string[] args)  
{  
    Console.WriteLine("Học bài ...");  
    Console.WriteLine("DONE");  
  
    Action<string> action = new Action<string>(ChoiGame);  
    action("Game cyber punk!");  
}
```

Delegate - ủy thác trong Java

- Js: callback
- Trước java 8: không hỗ trợ, dùng interface
- Java 8 hỗ trợ delegate

```
public private Runnable delegate;
// ...
this.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        delegate.run();
    }
});
// ...
if (someVar == someVal) {
    this.delegate = new Runnable() {
        @Override
        public void run() {
            showSomeScreen();
        }
    };
}
else {
    this.delegate = new Runnable() {
        @Override
        public void run() {
            showSomeOtherScreen();
        }
    };
}
```

Delegate - ủy thác trong Java (8)

- Đưa ra functional interface => định nghĩa function

```
@FunctionalInterface
public interface Functional<R,T> {
    R apply(T t);
}
```

```
@FunctionalInterface
public interface OrderProcessHandler {
    void handle(Order order);
}
```

```
@Service
@Slf4j
public class OrderServiceV2 {

    private final OrderProcessHandler orderProcessHandler;

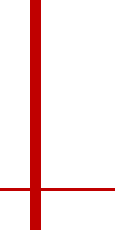
    @Autowired
    public OrderServiceV2(
        @Value("${orderService.writeToQueue}") boolean writeToQueueEnabled) {
        if (writeToQueueEnabled) {
            orderProcessHandler = this::saveBoth;
        } else {
            orderProcessHandler = this::persistToDb;
        }
    }

    public void process(Order order) {
        LocalDateTime entryTime = LocalDateTime.now();
        log.info("An order came at: " + entryTime);
        orderProcessHandler.handle(order);
    }

    private void saveBoth(Order order) {
        persistToDb(order);
        writeToQueue(order);
    }

    private void persistToDb(Order order){
        log.info("Persisting to Db...");
    }

    private void writeToQueue(Order order) {
        log.info("Writing to queue...");
    }
}
```

- 
- Delegate: tách biệt khai báo và code xử lý
 - cũng gần như interface
 - tuy nhiên delegate phục vụ cho xử lý theo sự kiện - event driven



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

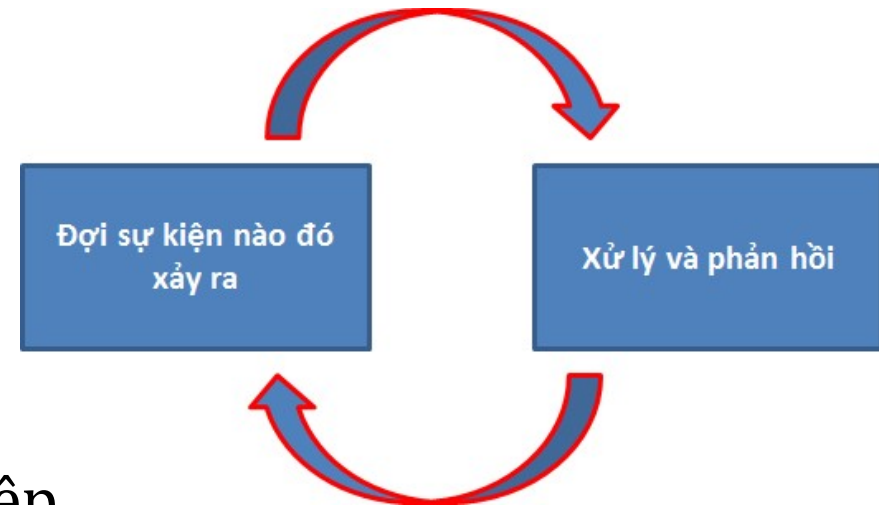
KHOA TOÁN – TIN

2. Sự kiện - Event



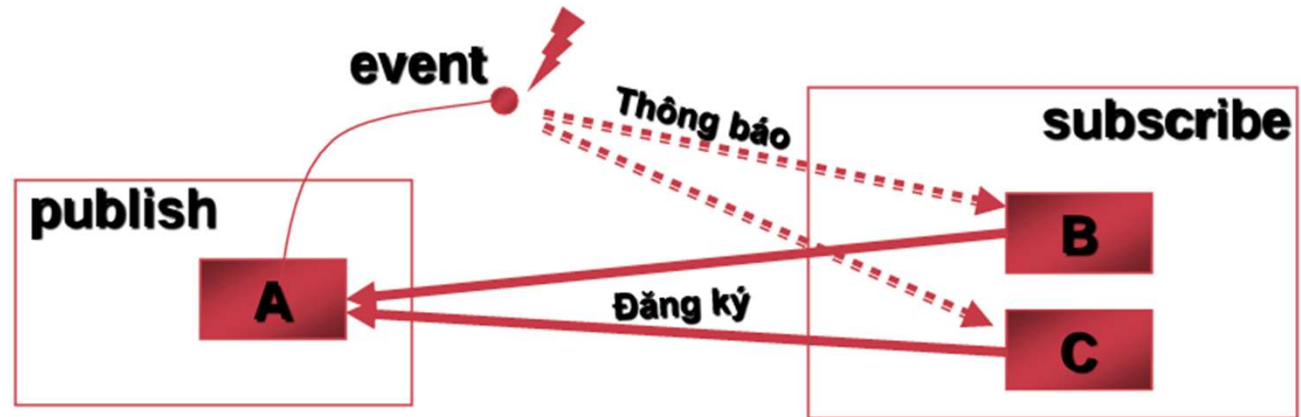
Event

- Lập trình hướng sự kiện
- Các đối tượng trao đổi qua thông điệp
- Sự kiện: một đối tượng thông báo việc xảy ra qua sự kiện => gửi tới đối tượng khác
- Đối tượng phát sinh sự kiện - Publisher
- Đối tượng nhận sự kiện - Subscriber



Event

- Ví dụ
 - Giao diện GUI
 - Ứng dụng web



- Button: phát sinh sự kiện Click
- Người dùng nhấn nút/button - sự kiện Click xảy ra
- Đoạn mã xử lý tương ứng được thực thi

Event

- Một số đặc điểm
 - Publisher: xác định event sẽ sinh ra >< Subscriber: xác định phản ứng
 - Event có thể được nhiều bên tiếp nhận và xử lý/phản ứng
 - Nếu event có nhiều bên tiếp nhận => các đoạn mã xử lý được gọi đồng thời

Event và Delegate

.Net:

- Event được xử lý qua cơ chế delegate
 - Lớp publishing định nghĩa delegate
 - Các lớp subscribing: method xử lý và đăng ký
 - Khi xảy ra event, method của lớp subscriber được gọi qua delegate
- Method xử lý sự kiện: gọi là event handler

Event và Delegate

- Trình xử lý sự kiện (event handler) trong .Net:
 - là void
 - có 2 đối số
 - đối số 1: đối tượng publisher - nguồn phát sinh sự kiện
 - đối số 2: đối tượng lớp dẫn xuất từ EventArgs

Event và Delegate

- Khai báo delegate xử lý sự kiện

```
public delegate void EventHandler<TEventArgs>(object sender, TEventArgs e);
```

- Khai báo sự kiện (publisher)

```
public event EventHandler<EventArgs> RaiseEvent;
```

- Xử lý sự kiện (subscriber)

- Đoạn mã (method) xử lý sự kiện và đăng ký delegate

Ví dụ 1

Kịch bản

- Publisher: Phát sinh sự kiện (vd sau mỗi 1 s)
- Subscriber: có 2 lớp đối tượng tiếp nhận và xử lý sự kiện
 - Lớp clockA: hiển thị thời gian theo mẫu "analog clock hh-> mm->"
 - Lớp clockB: hiển thị thời gian theo mẫu "digital clock hh:mm"

Ví dụ 1

- B1: Khai báo delegate
- B2: Xây dựng lớp publisher
 - khai báo event handler
 - method phát sinh sự kiện
- B3: xây dựng lớp tiếp nhận subscriber
 - Phương thức xử lý sự kiện
- B4: main: đăng ký tiếp nhận sự kiện, publisher phát sinh sự kiện

Ví dụ 1

■ B1: Khai báo delegate

```
//0. Data class
public class TimeInfor{
    DateTime time;
}
//1. Define a delegate
public delegate void TimeEventHandler(TimeInfor time);
```

Tên delegate

Đối tượng phát sinh

Ví dụ 1

- B2: Xây dựng lớp publisher
 - Khai báo event handler
 - method phát sinh sự kiện

```
public class TimeChangePublisher
{
    // Declare an event of type EventHandler
    public event TimeEventHandler OnTimeEvent;
    // ...
}
```

Ví dụ 1

■ B2: Lớp publisher

- Method phát sinh sự kiện

```
public class TimeChangePublisher
{
    public event TimeEventHandler OnTimeEvent;
    private System.Timers.Timer timer;
    public TimeChangePublisher() {
        timer = new System.Timers.Timer(2000);
        timer.Elapsed += TimerElapsed;
    }
    public void Start() { // Start the timer
        timer.Start();
    }
    public void Stop() { // Stop the timer
        timer.Stop();
    }
    // Method to raise the event
    private void TimerElapsed(object sender, ElapsedEventArgs e)
    {
        if (OnTimeEvent != null)
        {
            TimeInfor time = new TimeInfor();
            OnTimeEvent(time);
        }
    }
}
```

Ví dụ 1

- B3: xây dựng lớp tiếp nhận subscriber

```
public class AnalogSubscriber
{
    // Method that matches the delegate signature
    public void HandleEvent(TimeInfor time) {
        Console.WriteLine("ANALOG clock: {0}",
            time.time.ToString());
    }
}

public class DigitalSubscriber
{
    // Method that matches the delegate signature
    public void HandleEvent(TimeInfor time) {
        Console.WriteLine("DIGITAL clock: {0}",
            time.time.ToString());
    }
}
```

Ví dụ 1

■ Thực thi

```
TimeChangePublisher timeChangePublisher = new TimeChangePublisher();  
AnalogSubscriber analogSubscriber = new AnalogSubscriber();  
DigitalSubscriber digitalSubscriber = new DigitalSubscriber();
```

```
// Subscribe the HandleStudentAdmission methods to the  
OnStudentAdmitted event
```

```
timeChangePublisher.OnTimeEvent += analogSubscriber.HandleEvent;  
timeChangePublisher.OnTimeEvent += digitalSubscriber.HandleEvent;
```

```
// Start the publisher  
timeChangePublisher.Start();
```

```
// Keep the application running to observe the event handling  
Console.WriteLine("Press Enter to exit...");
```

```
Console.ReadLine();  
timeChangePublisher.Stop();
```

```
ANALOG clock: 10/15/2024 11:50:22 AM  
DIGITAL clock: 10/15/2024 11:50:22 AM  
ANALOG clock: 10/15/2024 11:50:24 AM  
DIGITAL clock: 10/15/2024 11:50:24 AM  
ANALOG clock: 10/15/2024 11:50:26 AM  
DIGITAL clock: 10/15/2024 11:50:26 AM  
ANALOG clock: 10/15/2024 11:50:28 AM  
DIGITAL clock: 10/15/2024 11:50:28 AM
```



BASIS FOR COMPARISON

DELEGATES

EVENTS

Basic	A delegate holds the reference of a method.	The event is an over-layered abstraction provided to the delegates.
Syntax	<code>delegate return_type delegate_name(parameter_list);</code>	<code>event event_delegate event_name;</code>
Keyword	A delegate is declared using a keyword "delegate."	An Event is declared using a keyword "event".
Declaration	A delegate is declared outside any class.	An event is declared inside a class.
Invoke	To invoke a method it has to be referred to the delegate.	To invoke a method it has to be assigned to the event.
Covariance and Contravariance	They provide flexibility to the delegates.	No such concept.
Event Accessor	No such concept.	Manages the list of the event handlers.
Dependency	Delegates are independent of events.	The event can not be created without delegates.